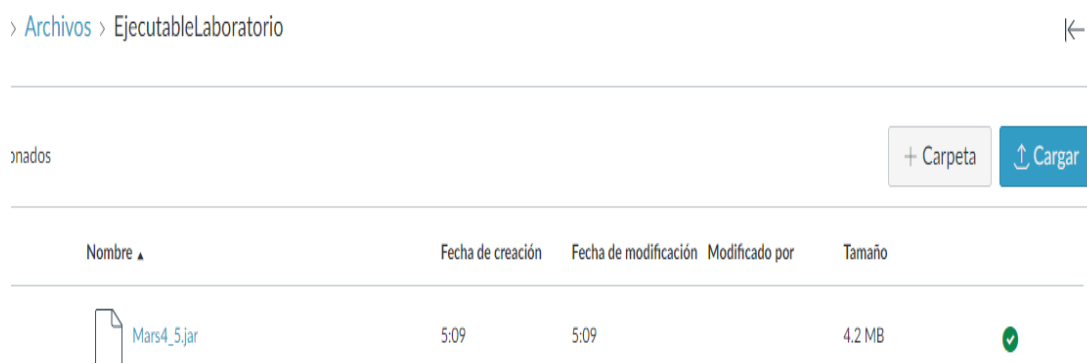


Asignatura	Datos del alumno	Fecha
Estructura de Computadores	Apellidos: López Fierro	
	Nombre: Marlon Fernando	

Laboratorio #1: Simulación y optimización de un programa en un procesador escalar segmentado

Preparación para el laboratorio

Para poder realizar la práctica de laboratorio deberá usarse el MARS (MIPS Assembler and Runtime Simulator), que básicamente es un IDE para MIPS. El archivo con el cual podrá ejecutar la actividad es un archivo Executable Jar File (.jar) que tiene como nombre: Mars4_5 y que se encuentra en la siguiente carpeta:



Dentro de las características de esta aplicación encontramos:

- Interfaz gráfica de usuario con control "apuntar y hacer clic" y editor integrado.
- Valores de registro y memoria fácilmente editables, similares a los de una hoja de cálculo.
- Visualización de valores en hexadecimal o decimal.
- Modo de línea de comandos para que los instructores prueben y evalúen fácilmente muchos programas.

Asignatura	Datos del alumno	Fecha
Estructura de Computadores	Apellidos: López Fierro	
	Nombre: Marlon Fernando	

- Registros de coma flotante, coprocesador1 y coprocesador2. Herramienta estándar: visualización y edición a nivel de bits de los registros de coma flotante de 32 bits (captura de pantalla).
- Ejecución en un solo paso a velocidad variable.
- Utilidad "Tool" para el control MIPS de dispositivos simulados. Herramienta estándar: Caché.

Adicionalmente, en la carpeta de documentación del curso se cargarán los siguientes documentos necesarios para la realización del laboratorio:

- Conceptos Básicos del Laboratorio: Arquitecturas Segmentadas (archivo PDF)
- Guía de Laboratorio: Optimización en Procesadores Segmentados (archivo PDF)
- Plantilla Informe (archivo MD)
- Tutorial MARS (archivo PDF)
- Guía Rápida de Instrucciones MIPS Assembler (archivo PDF)
- Archivo con el código Assembler (programa_base.asm)

Conceptos Previos

Assembler: es un lenguaje de programación de bajo nivel (también conocido como ensamblador) que se utiliza para escribir programas que se ejecutan directamente en un computador o en otros dispositivos electrónicos. Los programas escritos en ensamblador son traducidos a lenguaje de máquina, que es el lenguaje que entiende el procesador del computador.

A diferencia de los lenguajes de programación de alto nivel, como C++ o Java, que se enfocan en la abstracción y la simplificación del proceso de programación, el ensamblador es un lenguaje de programación muy cercano a la arquitectura del procesador. Esto significa que los programas escritos en ensamblador son muy

Asignatura	Datos del alumno	Fecha
Estructura de Computadores	Apellidos: López Fierro	
	Nombre: Marlon Fernando	

eficientes y rápidos, ya que se aprovechan al máximo las capacidades del procesador y se pueden controlar todos los detalles del hardware. Los archivos de este lenguaje tienen la extensión *.asm

Descripción del laboratorio

En esta práctica vamos a trabajar con un lenguaje de programación de bajo nivel llamado Assembler siguiendo cada uno de los pasos propuestos en el documento llamado Guia_laboratorio.pdf.

Entrega del laboratorio

Todos los lineamientos de entrega deben cumplirse para la revisión y evaluación de la actividad. Esto significa que, se debe llevar a cabo todos los pasos de la guía del laboratorio y presenta la actividad utilizando el archivo llamado Plantilla_Informe en formato PDF.

Lo primero que se debe realizar antes de hacer el envío de la actividad es cargar los dos scripts de la actividad en formato *.asm (programa_base y programa_optimizado) en GitHub, y compartir el enlace del repositorio en el documento llamado Plantilla_Informe para validar el código de la actividad. Es importante que el nombre del repositorio tenga la siguiente nomenclatura: nombre-apellido-estructura-computadores-act01. Cada uno de los scripts debe ser presentado en archivos independientes. **Para que sea evaluada la actividad los scripts deben estar cargados en GitHub y la actividad presentada utilizando la Plantilla_informe en formato PDF.**

- ▶ Nota: si la actividad la realizan de forma grupal todos los integrantes debe publicar el enlace a su respectivo perfil en GitHub e incluir los enlaces al final del informe.
- ▶ Cada una de las líneas debe estar comentada con la respectiva descripción de que realiza cada instrucción.

Asignatura	Datos del alumno	Fecha
Estructura de Computadores	Apellidos: López Fierro	
	Nombre: Marlon Fernando	

Rúbrica de Evaluación

Criterio Evaluación	Descripción	Porcentaje
Análisis del código base	Identificación de riegos y explicación técnica	20%
Optimización del código	Aplica técnicas de optimización eliminando stalls	30%
Resultados y métricas	Tablas comparativas con datos correctos del cálculo	20%
Justificación técnica	Explicación técnica del proceso de optimización aplicado	20%
Formato y presentación	Utiliza plantilla informe y cargue código Github	10%

RESOLUCION DEL TALLER:

Enlace del repositorio: <https://github.com/fierromarlonfernando-code/colgii26-taller-5-estructura>

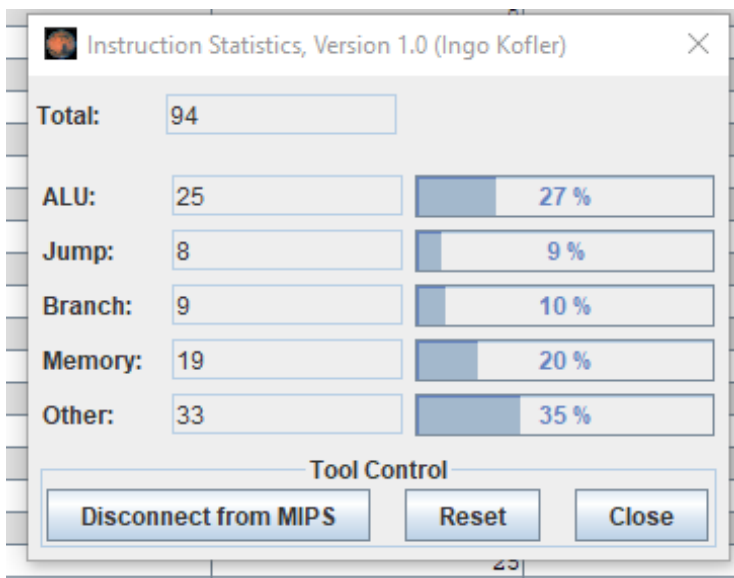
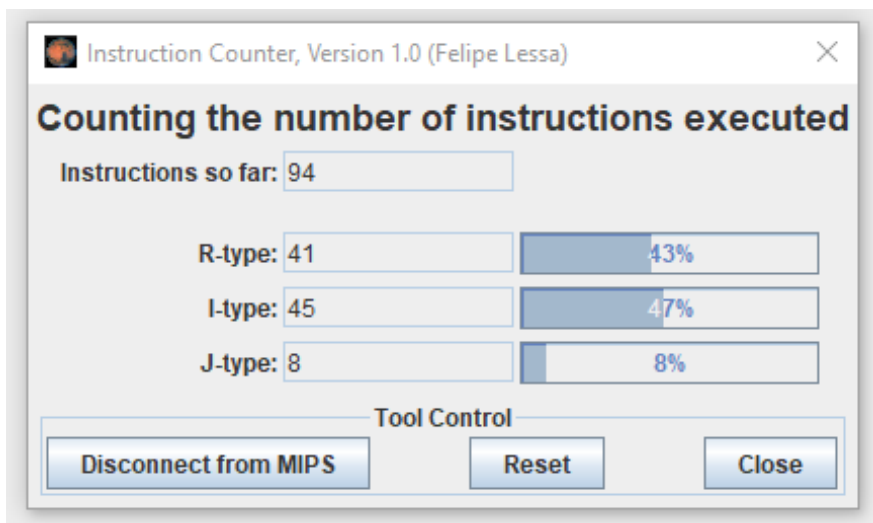
1. ANÁLISIS DEL CÓDIGO BASE

El programa calcula: $Y[i] = A \times X[i] + B$ para un vector de 8 elementos usando MIPS.

Datos iniciales:

- Vector X: [1, 2, 3, 4, 5, 6, 7, 8]
- Constante A: 3
- Constante B: 5
- Tamaño: 8 elementos

Asignatura	Datos del alumno	Fecha
Estructura de Computadores	Apellidos: López Fierro	
	Nombre: Marlon Fernando	



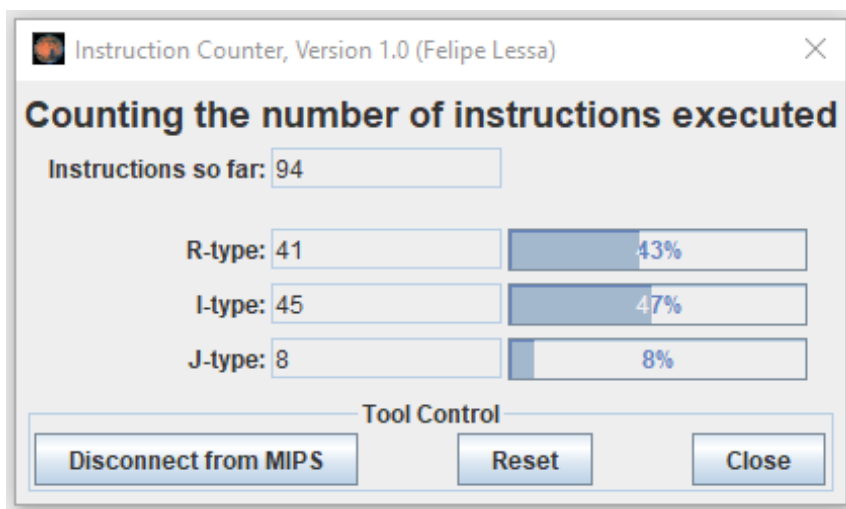
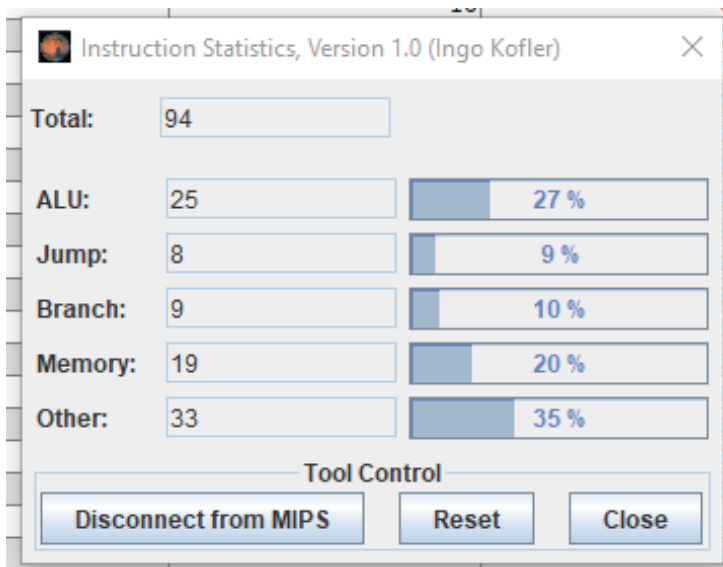
2. RIESGOS IDENTIFICADOS (HAZARDS)

- Stalls estimados: 16 (8 iteraciones × 2 stalls/iteración)
- Ciclos estimados: 110
- CPI: 1.17

Asignatura	Datos del alumno	Fecha
Estructura de Computadores	Apellidos: López Fierro	
	Nombre: Marlon Fernando	

CÓDIGO OPTIMIZADO:

- Total de Instrucciones: 94
- Stalls estimados: 8 (solo dependencia mul-addu)
- Ciclos estimados: 102
- CPI: 1.09



Asignatura	Datos del alumno	Fecha
Estructura de Computadores	Apellidos: López Fierro	
	Nombre: Marlon Fernando	

MEJORA ALCANZADA:

- Reducción de stalls: 50%
- Reducción de ciclos: 7.3%
- Mejora en CPI: 6.8%

5. CONCLUSIONES

El reordenamiento de instrucciones independientes es una técnica fundamental en la optimización de código para procesadores segmentados.

El análisis teórico demuestra que aunque MARS es un simulador funcional, las mejoras identificadas tendrían impacto directo en un procesador real.

La dependencia mul-addu no puede eliminarse completamente, pero la optimización del Load-Use hazard demuestra el impacto de comprender la arquitectura del procesador en la programación de bajo nivel.